

FECAP

Análise e Desenvolvimento de Sistemas

PROJETO INTERDISCIPLINAR - KFKA

Banco de Dados - Entrega 1

Criação e justificativa de índices para melhoria de performance das consultas

São Paulo - 2026

1. Objetivo da entrega

Este documento apresenta os índices adicionais criados no banco PostgreSQL do KFKA, a plataforma de acompanhamento escolar que estamos desenvolvendo no projeto interdisciplinar. A ideia foi olhar para as consultas que a aplicação mais executa e criar índices que ajudem essas buscas. Para cada índice estão descritos o comando de criação, a consulta que ele atende e o motivo de ter sido criado.

O banco já tinha 31 índices antes desta etapa, todos gerados automaticamente pelas chaves primárias (PRIMARY KEY) e pelas restrições de unicidade (UNIQUE). Esses índices garantem a integridade dos dados e resolvem bem as buscas feitas pela chave, mas não cobrem todas as consultas da aplicação, principalmente as que filtram por colunas que não são a primeira da chave. Partindo disso, criei oito índices novos.

2. Como escolhi os índices

- Primeiro levantei os índices que já existiam, para não criar nada repetido em cima de PRIMARY KEY e UNIQUE.
- Depois separei as consultas mais importantes do sistema: acompanhamento por aluno, histórico, vínculos do professor, acesso do responsável e relatórios por tag.
- Nos índices compostos, coloquei as colunas na mesma ordem em que aparecem nos filtros da consulta, porque o B-tree só aproveita as colunas a partir da primeira.
- Usei o tipo B-tree, que é o padrão do PostgreSQL e atende bem as comparações por igualdade e as ordenações deste projeto.
- Por último, rodei EXPLAIN ANALYZE em cada consulta para ver como o banco decidiu executá-la.

3. Índices criados

A tabela abaixo resume os oito índices. Em seguida cada um deles é detalhado.

Nº	Nome do índice	Tabela	Colunas
----	----------------	--------	---------

Nº	Nome do índice	Tabela	Colunas
1	idx_acompanhamento_aluno_bimestre_status	acompanhamento	id_aluno, id_bimestre, status
2	idx_historico_acompanhamento_data	historico_acompanhamento	id_acompanhamento, data_hora
3	idx_tdp_professor_ativo	turma_disciplina_professor	id_professor, ativo
4	idx_aluno_responsavel_responsavel_autorizado	aluno_responsavel	id_responsavel, autorizado
5	idx_acompanhamento_tag_tag	acompanhamento_tag	id_tag, id_acompanhamento
6	idx_acompanhamento_vinculo_bimestre	acompanhamento	id_vinculo, id_bimestre
7	idx_turma_aluno_aluno_ano	turma_aluno	id_aluno, id_ano_letivo
8	idx_ciencia_acompanhamento_responsavel	ciencia_acompanhamento	id_acompanhamento, id_responsavel

Tabela 1 - Resumo dos índices criados.

3.1 idx_acompanhamento_aluno_bimestre_status

```
CREATE INDEX idx_acompanhamento_aluno_bimestre_status
ON acompanhamento (id_aluno, id_bimestre, status);
```

Uso principal: Uma das telas mais usadas do KFKA mostra os acompanhamentos de um aluno dentro de um bimestre. Na maioria das vezes a consulta ainda filtra pelo status, principalmente PUBLICADO, que é o que o responsável tem permissão de ver.

Justificativa: As três colunas do filtro estão no mesmo índice e na mesma ordem em que aparecem na consulta. Nenhum dos índices que já existiam cobria essa combinação, então essa busca dependeria de varrer a tabela inteira. Como acompanhamento é a tabela que mais vai crescer no sistema, esse foi o primeiro índice que resolvi criar.

3.2 idx_historico_acompanhamento_data

```
CREATE INDEX idx_historico_acompanhamento_data
ON historico_acompanhamento (id_acompanhamento, data_hora);
```

Uso principal: O histórico é consultado por acompanhamento e precisa aparecer em ordem cronológica, já que serve para auditoria e rastreabilidade das alterações.

Justificativa: A consulta filtra por id_acompanhamento e depois ordena por data_hora, então montei o índice exatamente nessa ordem. Quando o PostgreSQL

usa esse índice, ele já entrega as linhas ordenadas e não precisa executar um Sort depois, o que economiza trabalho principalmente em acompanhamentos com muitas alterações registradas.

3.3 idx_tdp_professor_ativo

```
CREATE INDEX idx_tdp_professor_ativo  
ON turma_disciplina_professor (id_professor, ativo);
```

Uso principal: No módulo do professor o sistema precisa saber em quais turmas e disciplinas aquele professor pode lançar acompanhamento, considerando só os vínculos ativos.

Justificativa: Já existia um índice UNIQUE nessa tabela, mas ele começa por id_turma e id_disciplina. Um índice B-tree só é aproveitado quando a busca usa as primeiras colunas dele, e aqui a consulta parte do id_professor. Por isso criei um índice próprio começando pela coluna que o módulo realmente usa como filtro, junto com o campo ativo.

3.4 idx_aluno_responsavel_responsavel_autorizado

```
CREATE INDEX idx_aluno_responsavel_responsavel_autorizado  
ON aluno_responsavel (id_responsavel, autorizado);
```

Uso principal: Quando o responsável entra no sistema, a primeira coisa que a aplicação faz é descobrir quais alunos estão vinculados a ele e quais desses vínculos estão autorizados.

Justificativa: A chave primária da tabela começa por id_aluno, o que resolve o caminho contrário (a partir do aluno, quem são os responsáveis). Nesse caso o ponto de partida é o responsável, então o índice novo inverte a ordem e ainda inclui o campo autorizado, que entra no filtro por ser uma regra de acesso.

3.5 idx_acompanhamento_tag_tag

```
CREATE INDEX idx_acompanhamento_tag_tag  
ON acompanhamento_tag (id_tag, id_acompanhamento);
```

Uso principal: Os relatórios da coordenação filtram acompanhamentos por tag, como Recuperação, Atenção ou Boa evolução.

Justificativa: A chave primária de acompanhamento_tag começa por id_acompanhamento, que serve para listar as tags de um acompanhamento

específico. O relatório faz o caminho oposto: escolhe a tag e busca todos os acompanhamentos ligados a ela. O índice foi criado com `id_tag` na frente justamente para atender esse sentido da consulta.

3.6 `idx_acompanhamento_vinculo_bimestre`

```
CREATE INDEX idx_acompanhamento_vinculo_bimestre  
ON acompanhamento (id_vinculo, id_bimestre);
```

Uso principal: Professores e relatórios acadêmicos consultam os acompanhamentos de um vínculo de turma, disciplina e professor dentro de um bimestre.

Justificativa: O `id_vinculo` representa a combinação de turma, disciplina, professor e ano letivo, e é por ele que o professor enxerga suas próprias turmas. Como quase sempre esse filtro vem acompanhado do bimestre, juntei as duas colunas em um índice só, em vez de deixar o banco filtrar uma delas depois da leitura.

3.7 `idx_turma_aluno_aluno_ano`

```
CREATE INDEX idx_turma_aluno_aluno_ano  
ON turma_aluno (id_aluno, id_ano_letivo);
```

Uso principal: O sistema precisa descobrir em qual turma o aluno está matriculado em determinado ano letivo. Essa informação também é usada nas validações de integridade do acompanhamento.

Justificativa: A chave primária de `turma_aluno` começa por `id_turma`, que atende a listagem de alunos de uma turma. Várias consultas fazem o contrário e partem do aluno mais o ano letivo, então criei o índice nessa ordem para cobrir esse caso.

3.8 `idx_ciencia_acompanhamento_responsavel`

```
CREATE INDEX idx_ciencia_acompanhamento_responsavel  
ON ciencia_acompanhamento (id_acompanhamento, id_responsavel);
```

Uso principal: Antes de exibir um acompanhamento publicado, o sistema verifica se aquele responsável já registrou ciência.

Justificativa: A verificação usa as duas colunas ao mesmo tempo e espera no máximo um registro como resposta. Com o índice cobrindo exatamente esse par, o banco chega direto na linha em vez de percorrer a tabela procurando a combinação.

4. Validação com EXPLAIN ANALYZE

As oito consultas foram testadas com EXPLAIN ANALYZE. Como a base de testes ainda tem poucos registros, na maioria dos casos o PostgreSQL escolheu Seq Scan, ou seja, preferiu ler a tabela inteira. Em tabela pequena isso costuma sair mais barato do que abrir o índice e depois buscar as linhas correspondentes.

Então o Seq Scan que aparece nos testes não quer dizer que os índices estejam errados. Eles foram feitos para consultas que o sistema realmente executa e a tendência é que passem a ser usados conforme a base crescer. Em alguns testes desativei o Seq Scan temporariamente, só para confirmar que o banco conseguia usar o índice criado, e depois voltei a configuração ao normal.

Nº	Tabela	Registros	Plano padrão	Retornadas / removidas	Execução
1	acompanhamento	40	Seq Scan	1 / 39	0,034 ms
2	historico_acompanhamento	146	Seq Scan + Sort	4 / 142	0,046 ms
3	turma_disciplina_professor	12	Seq Scan	3 / 9	0,028 ms
4	aluno_responsavel	12	Seq Scan	2 / 10	0,034 ms
5	acompanhamento_tag	69	Seq Scan	21 / 48	0,039 ms
6	acompanhamento	40	Seq Scan	2 / 38	0,040 ms
7	turma_aluno	12	Seq Scan	1 / 11	0,046 ms
8	ciencia_acompanhamento	15	Seq Scan	1 / 14	0,385 ms

Tabela 2 - Resultado do EXPLAIN ANALYZE de cada consulta.

Sobre os tempos: a base de testes é pequena, então esses valores servem só como referência desta etapa e não como medida de desempenho real. O que interessava no EXPLAIN ANALYZE era enxergar o plano escolhido para cada consulta e conferir se os índices criados fazem sentido para elas.

4.1 Evidências do uso dos índices

No primeiro exemplo o Seq Scan foi desativado temporariamente só para a demonstração. Nessa condição o PostgreSQL usou direto o índice composto:

	QUERY PLAN text
1	Index Scan using idx_acompanhamento_aluno_bimestre_status on acompanhamento (cost=0.14..8.16 rows=1 width=170) (actual time=0.023..0.024 rows=1.00 lo...
2	Index Cond: ((id_aluno = 4) AND (id_bimestre = 1) AND ((status)::text = 'PUBLICADO'::text))
3	Index Searches: 1
4	Buffers: shared hit=2
5	Planning Time: 0.106 ms
6	Execution Time: 0.043 ms

Figura 1 - Index Scan utilizando idx_acompanhamento_aluno_bimestre_status.

No índice do histórico fiz o mesmo teste controlado. A consulta por id_acompanhamento com ORDER BY data_hora rodou sem nenhuma etapa de Sort, o que mostra que a ordem das colunas do índice combina com a ordenação pedida pela consulta:

	QUERY PLAN text
1	Index Scan using idx_historico_acompanhamento_data on historico_acompanhamento (cost=0.14..11.71 rows=4 width=85) (actual time=0.026..0.027 rows=4.00 lo...
2	Index Cond: (id_acompanhamento = 4)
3	Index Searches: 1
4	Buffers: shared hit=2
5	Planning Time: 0.101 ms
6	Execution Time: 0.045 ms

Figura 2 - Index Scan utilizando idx_historico_acompanhamento_data, sem Sort separado.

5. Conclusão

Foram criados oito índices adicionais, pensados nas consultas mais usadas do KFKA. Eles complementam os índices de PRIMARY KEY e UNIQUE que já existiam e atendem buscas dos módulos de professor, responsável e administração, além do histórico, dos relatórios por tag e da confirmação de ciência.

Os testes com EXPLAIN ANALYZE mostraram que, com a quantidade atual de dados, o PostgreSQL costuma preferir Seq Scan, o que já era esperado em tabelas pequenas. Nos testes controlados com Index Scan deu para confirmar que os índices podem ser aproveitados nas consultas para as quais foram planejados. Conforme o banco for acumulando registros, esses índices devem evitar leituras desnecessárias e melhorar o tempo de resposta das consultas.

Anexo A - Script completo de criação dos índices

```
CREATE INDEX idx_acompanhamento_aluno_bimestre_status  
ON acompanhamento (id_aluno, id_bimestre, status);
```

```
CREATE INDEX idx_historico_acompanhamento_data  
ON historico_acompanhamento (id_acompanhamento, data_hora);
```

```
CREATE INDEX idx_tdp_professor_ativo  
ON turma_disciplina_professor (id_professor, ativo);
```

```
CREATE INDEX idx_aluno_responsavel_responsavel_autorizado  
ON aluno_responsavel (id_responsavel, autorizado);
```

```
CREATE INDEX idx_acompanhamento_tag_tag  
ON acompanhamento_tag (id_tag, id_acompanhamento);
```

```
CREATE INDEX idx_acompanhamento_vinculo_bimestre  
ON acompanhamento (id_vinculo, id_bimestre);
```

```
CREATE INDEX idx_turma_aluno_aluno_ano  
ON turma_aluno (id_aluno, id_ano_letivo);
```

```
CREATE INDEX idx_ciencia_acompanhamento_responsavel  
ON ciencia_acompanhamento (id_acompanhamento, id_responsavel);
```